A MINI PROJECT REPORT ON

"GPU-Accelerated Glitch Art Studio"

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY,
PUNE IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE
OF

Computer Engineering

SUBMITTED BY
(Class –BE A, Batch – B1)
Shashank Gosavi
Taslim Ansari
Karunya Chavan
Shruti Bachal
UNDER THE GUIDANCE OF
Mrs. Jayprabha Kanse

COMPUTER ENGINEERING
P.E.S MODERN COLLEGE OF ENGINEERING
PUNE – 411005.
SAVITRIBAI PHULE PUNE UNIVERSITY

[2024-25]

Progressive Education Society's
Modern College of Engineering
Shivajinagar, Pune : 411005.

CERTIFICATE

This is to certify that the following students of Computer Engineering of PES's, Modern College of Engineering have successfully completed their mini project and design of project entitled "**GPU-Accelerated Glitch Art Studio**" under the guidance of the course instructor.

The Group Members are:

Roll Number	Name
41027	Shashank Gosavi
41004	Taslim Ansari
41013	Karunya Chavan
41003	Shruti Bachal

This is in partial fulfillment of the award of the degree Computer Engineering of Savitribai Phule Pune University.

Date:

Mrs. Jayprabha Kanse
Course Instructor

Prof. Dr. Mrs. S. A. Itkar
H.O.D
Department of Computer Engineering

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Mrs. Jayprabha Kanse**, our project guide, for her consistent guidance, technical insights, and constructive feedback throughout the development of our mini project, **"GPU-Accelerated Glitch Art Studio"** Her expertise in web technologies has been instrumental in shaping the project and ensuring its successful completion.

We are also thankful to **Prof. Dr. Mrs. S. A. Itkar**, Head of the Department, for providing us with the academic resources and a conducive environment that enabled us to undertake and complete this project effectively.

We acknowledge the support of the **Department of Computer Engineering, Progressive Education Society's Modern College of Engineering**, for facilitating access to the required tools and infrastructure during this project.

Lastly, we extend our appreciation to all faculty members and peers whose timely inputs and suggestions contributed to the improvement and refinement of our work.

Table of Contents

ABSTRACT 1
Introduction 2
1.1 Overview: 3
Objectives 4
Motivation 6
3.1 Motivation: 7
Scope 8
System 10
Architecture 10
System 12
Specifications 12
Results 15
Conclusion 18
References 20

ABSTRACT

The "GPU-Accelerated Glitch Art Studio" is a high-performance web application designed to help digital artists and enthusiasts create stunning, studio-grade glitch art. Leveraging the immense parallel processing power of NVIDIA GPUs through CUDA, this system allows users to selectively sort pixels in real-time based on advanced spatial masking and multi-channel thresholding (Hue, Saturation, Lightness, and RGB).

The application features a modern React/Vite frontend for an intuitive, interactive user experience and a robust FastAPI Python backend that dynamically compiles and executes native CUDA C++ kernels using CuPy. This architecture ensures real-time feedback and high-throughput image manipulation, eliminating the bottlenecks typically associated with CPU-based image processing tools.

This project demonstrates core High-Performance Computing (HPC) concepts, such as parallel algorithms, thread synchronization, memory management, and GPU acceleration, providing a practical example of how parallel computing technologies can be applied to build creative and responsive multimedia applications.

1. Introduction

1.1 Overview:

In the digital art and graphic design space, glitch art has emerged as a popular aesthetic. However, generating complex glitch effects like pixel sorting on high-resolution images is computationally intensive. The GPU-Accelerated Glitch Art Studio is an HPC-driven web application designed to simplify and accelerate this process. By utilizing CUDA, the system processes entire rows or columns of pixels simultaneously, offering instantaneous previews and rendering.

1.2 Problems in Manual/Traditional Systems:

Traditional CPU-based methods of pixel sorting are extremely slow, often taking several seconds or minutes to process a single high-resolution image. Traditional applications process pixels sequentially, leading to high latency, poor interactivity, and a frustrating user experience. Furthermore, many existing tools lack advanced spatial masking or multi-channel sorting, restricting the creative freedom of digital artists.

1.3 Proposed Solution:

To overcome the limitations of CPU-bound processing, the proposed GPU-Accelerated Glitch Art Studio utilizes parallel computing technologies to deliver a responsive, fast, and feature-rich platform. The system employs a native CUDA C++ kernel to perform in-place parallel Quick Sort across thousands of GPU threads simultaneously. Its modern web architecture ensures ease of use, while the backend provides a seamless creative workflow.

1.4 Use Case Scenarios:

The GPU-Accelerated Glitch Art Studio is a versatile solution designed for artists, designers, and hobbyists. For digital artists, the system offers the ability to import high-resolution media, apply selective spatial masking using interactive bounding boxes, and sort pixels based on various channels like Hue or Lightness. This allows for fine-grained control over the final glitch aesthetic.

For developers and HPC students, the system serves as an excellent educational tool, demonstrating the practical integration of low-level CUDA kernels with modern high-level web frameworks. The real-time visual feedback makes understanding parallel processing concepts much more intuitive. The system’s architecture guarantees extremely fast execution times and cross-platform accessibility.

2. Objectives

2.1 Functional Objectives:

- Allow users to import high-resolution images securely into the application.
- Enable users to apply parallel pixel sorting using different sorting channels (RGB, HSL).
- Provide interactive bounding box masking to apply glitches to specific spatial regions.
- Offer real-time visual feedback and instantaneous rendering previews.
- Support filtering and sorting based on dynamic user-defined threshold values.
- Generate downloadable high-quality JPEG exports of the final art.
- Provide multi-directional sorting (Left-to-Right, Top-to-Bottom, etc.).
- Ensure responsive design for access across desktop and modern web browsers.

2.2 Technical Objectives:

- Implement a high-performance backend using Python FastAPI and CuPy.
- Develop native CUDA C++ kernels for parallel execution of Quick Sort algorithms.
- Ensure high throughput and low latency by utilizing GPU shared and global memory effectively.
- Integrate a modern React and Vite frontend for a seamless user interface.
- Utilize multipart/form-data for efficient image transfer between frontend and backend.
- Implement dynamic NVRTC (NVIDIA Runtime Compilation) via CuPy to avoid complex C++ build systems.
- Ensure fast memory transfers between Host (CPU) and Device (GPU).
- Deploy the frontend application using standard Node.js development servers.

3. Motivation

3.1 Motivation:

- The increasing demand for high-resolution digital art and fast rendering in the creative industry.
- Traditional CPU methods are time-consuming and lack real-time interactivity.
- Many users struggle with slow software when applying complex image transformations.
- GPU Computing offers highly parallelized, scalable performance for multimedia applications.
- There is a growing need for web-based tools that leverage native hardware acceleration.
- Visual programming tools and real-time previews greatly enhance the artistic workflow.
- Encouraging exploration of High-Performance Computing concepts among students.
- Demonstrating the power of NVIDIA GPUs beyond gaming and AI, into creative domains.
- Increased awareness of modern web architectures bridging low-level code with UI.
- Opportunity to apply parallel computing concepts in a real-world, visually impactful project.
- Lack of centralized, open-source platforms combining custom glitch algorithms and web UIs.
- Rising popularity of the glitch aesthetic highlights a growing user base.
- Many existing apps are either command-line only or lack flexibility for different sorting needs.
- CUDA-based solutions reduce processing time by orders of magnitude.
- Growing importance of interactive, responsive digital art studios.
- Encouraging the habit of optimizing algorithms, especially among computer engineering students.

4.

Scope

4.1 Scope:

- The system will allow users to launch a local web server and access the studio via a browser.
- Users can upload custom images and apply GPU-accelerated pixel sorting in real-time.
- The application will support threshold management, allowing users to define exactly which pixels are sorted.
- Users will be able to view their glitch art through a dynamic, zoomable canvas.
- The system will support spatial masking via interactive bounding boxes drawn by the user.
- All computationally heavy tasks will be offloaded to the NVIDIA GPU via native CUDA kernels.
- The platform will offer downloadable JPEG images for the final exported artwork.
- It will support multiple sorting styles including Hue, Saturation, Lightness, and independent color channels.
- The application will be accessible through modern web browsers.
- **Image processing will be performed entirely in-memory, ensuring fast I/O and privacy.**
- The system is designed to be scalable, allowing for the addition of new CUDA kernels for different effects.
- The system will include directional sorting (Left to Right, Top to Bottom).
- Users will receive visual loading indicators during processing.
- A responsive control panel will be developed for modifying parameters interactively.
- The system will include error handling for invalid files or incompatible GPU drivers.
- The architecture will be designed with modular components (Frontend, Backend, Kernel).

5. System Architecture

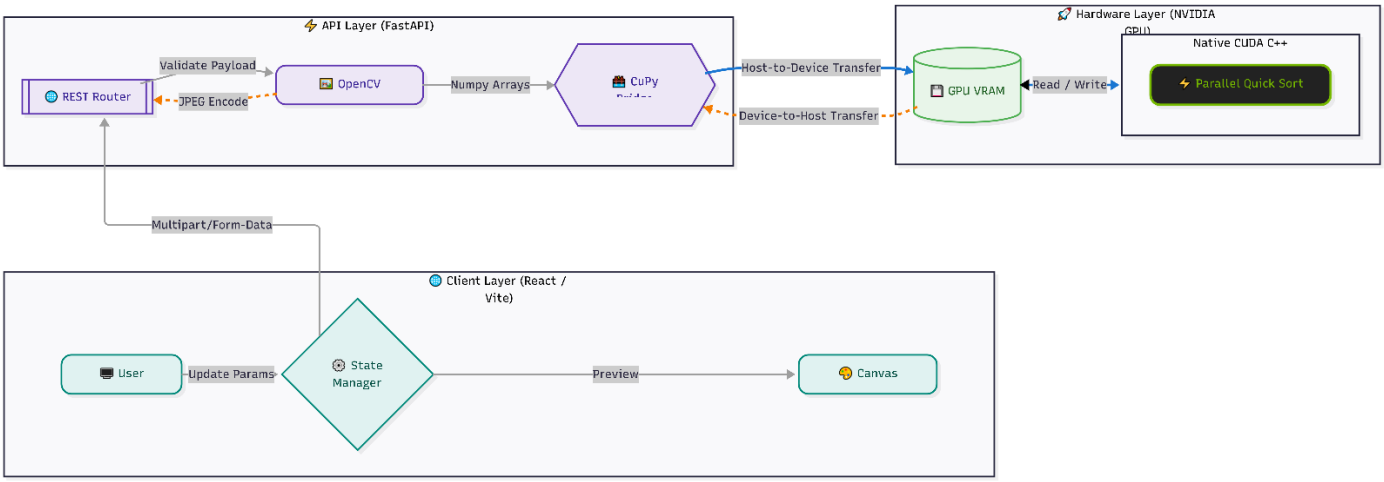


Fig. 5.1 Architecture Diagram

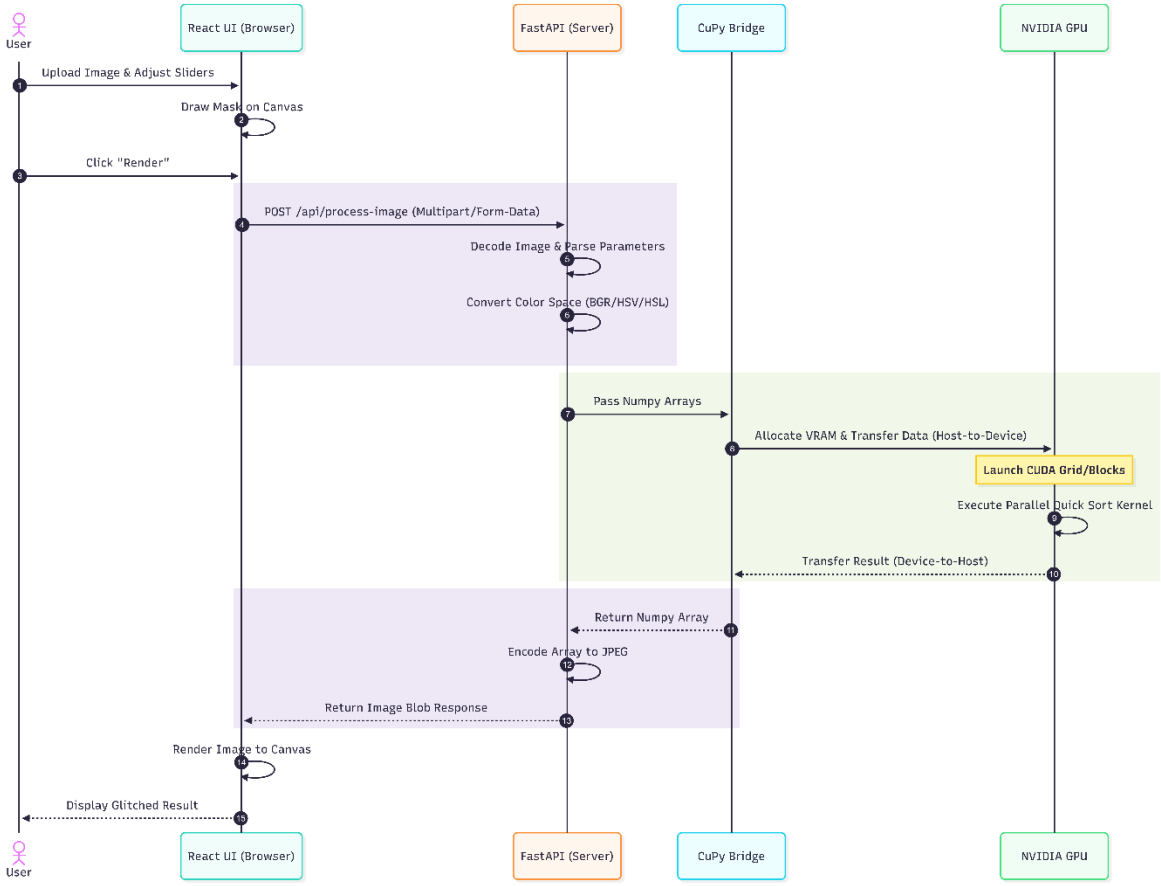


Fig: 5.2 Sequence Diagram (Timing & Data Flow)

6. System Specifications

6.1 Functional Requirements:

- User interface for parameter adjustment and image uploading.
- Add, edit, and apply interactive spatial masks.
- Set and manage threshold limits dynamically.
- Generate and view glitch art on a responsive canvas.
- Select between multiple sorting channels (Hue, Lightness, RGB).
- Real-time communication between React frontend and FastAPI backend.
- Downloadable art exports in JPEG format.
- Support for various image resolutions.

6.2 Non-Functional Requirements:

- High throughput and low latency using GPU acceleration.
- Secure local processing ensuring user data privacy.
- Scalable kernel design to support 4K and 8K image resolutions.
- Fast response times optimized via CUDA parallel execution.
- User-friendly interface with responsive studio design.
- Efficient memory management to prevent VRAM overflow.
- Cross-platform compatibility via standard web browsers.

6.3 Optional Enhancements:

- AI-based aesthetic recommendations for optimal glitch parameters.
- Integration with video processing for animated glitch art.
- Support for multi-GPU setups for even faster rendering.
- Real-time webcam feed pixel sorting.
- Additional CUDA effects like databending or chromatic aberration.
- Advanced timeline features for keyframing parameters.

6.4 Hardware Requirements:

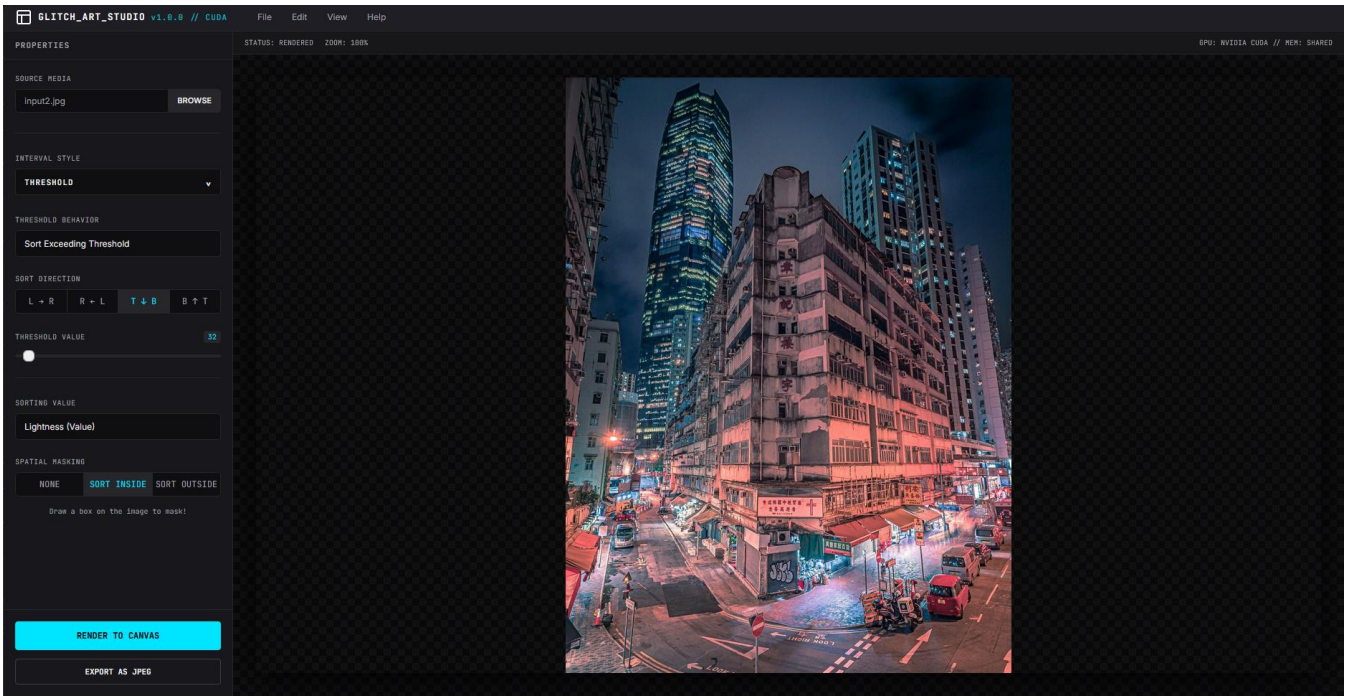
Component	Minimum Requirement
Processor	Intel Core i5 or higher
RAM	8 GB
Hard Disk	256 GB HDD or SDD
Internet	Stable internet connection
Devices	PC, Laptop, Tablet, Smartphone

6.5 Software Requirements:

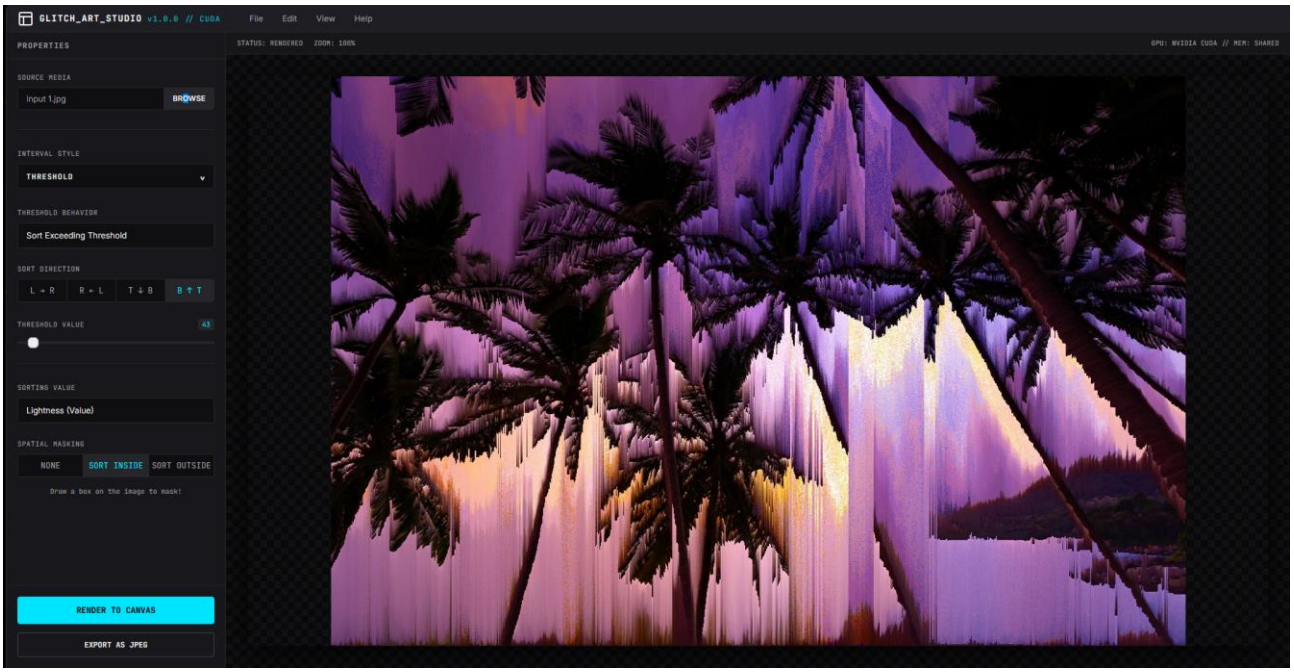
Category	Technology/Tool
Development Tools	Visual Studio Code, Git, Postman
Frontend	React JS
Cloud Services	Netlify
Browser Support	Chrome, Firefox, Edge, Safari
Operating Systems	Windows, macOS, Linux

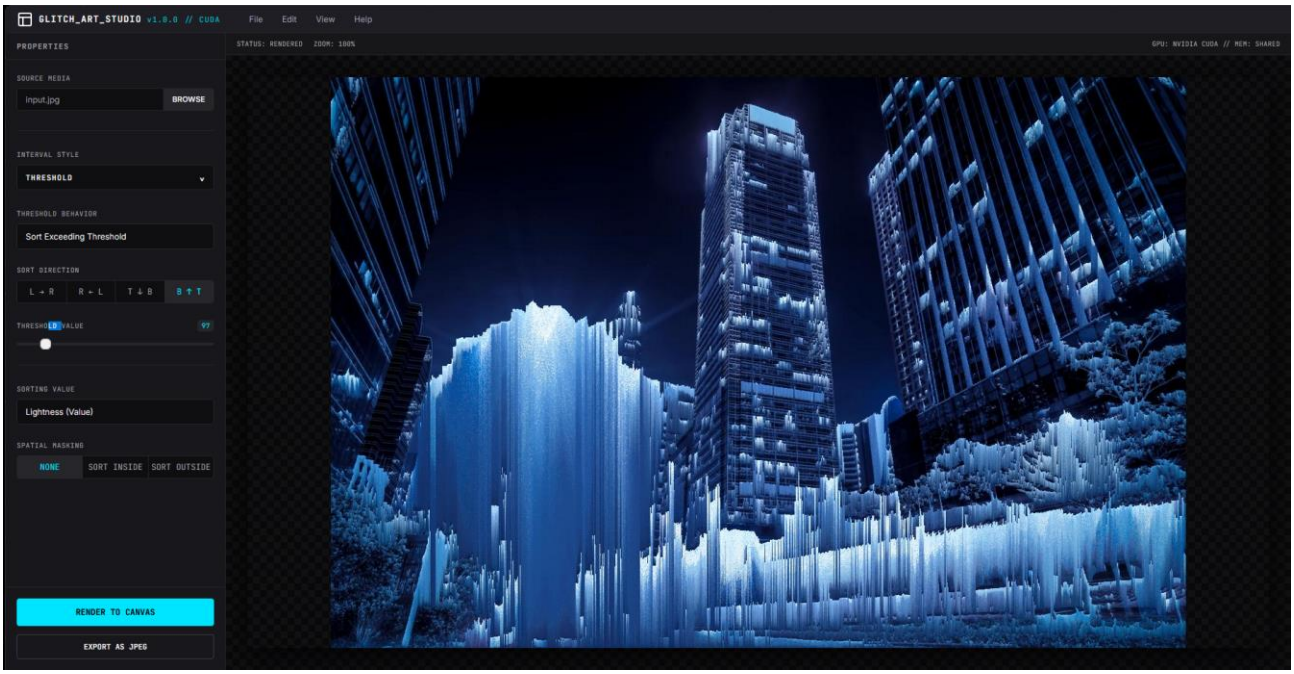
7.

Results


GLITCH_ART_STUDIO v1.0.0 // CUDA
File
Edit
View
Help
PROPERTIES
STATUS: RENDERED ZOOM: 100%
GPU: NVIDIA CUDA // MEM: SHARED
SOURCE MEDIA
input2.jpg
BROWSE
INTERVAL STYLE
THRESHOLD
THRESHOLD BEHAVIOR
Sort Exceeding Threshold
SORT DIRECTION
L → R
R ← L
T ↓ B
B ↑ T
THRESHOLD VALUE
32
SORTING VALUE
Lightness (Value)
SPATIAL MASKING
NONE
SORT INSIDE
SORT OUTSIDE
Draw a box on the image to mask!
RENDER TO CANVAS
EXPORT AS JPEG

GLITCH_ART_STUDIO v1.0.0 // CUDA
File
Edit
View
Help
PROPERTIES
STATUS: RENDERED ZOOM: 100%
GPU: NVIDIA CUDA // MEM: SHARED
SOURCE MEDIA
input2.jpg
BROWSE
INTERVAL STYLE
THRESHOLD
THRESHOLD BEHAVIOR
Sort Exceeding Threshold
SORT DIRECTION
L → R
R ← L
T ↓ B
B ↑ T
THRESHOLD VALUE
32
SORTING VALUE
Lightness (Value)
SPATIAL MASKING
NONE
SORT INSIDE
SORT OUTSIDE
Draw a box on the image to mask!
RENDER TO CANVAS
EXPORT AS JPEG


GLITCH_ART_STUDIO v1.0.0 // CUDA
File
Edit
View
Help
PROPERTIES
STATUS: RENDERED ZOOM: 100%
GPU: NVIDIA CUDA // MEM: SHARED
SOURCE MEDIA
input 1.jpg
BROWSE
INTERVAL STYLE
THRESHOLD
THRESHOLD BEHAVIOR
Sort Exceeding Threshold
SORT DIRECTION
L → R
R ← L
T ↓ B
B ↑ T
THRESHOLD VALUE
43
SORTING VALUE
Lightness (Value)
SPATIAL MASKING
NONE
SORT INSIDE
SORT OUTSIDE
Draw a box on the image to mask!
RENDER TO CANVAS
EXPORT AS JPEG


GLITCH_ART_STUDIO v1.0.0 // CUDA
File
Edit
View
Help
PROPERTIES
STATUS: RENDERED ZOOM: 100%
GPU: NVIDIA CUDA // MEM: SHARED
SOURCE MEDIA
input.jpg
BROWSE
INTERVAL STYLE
THRESHOLD
THRESHOLD BEHAVIOR
Sort Exceeding Threshold
SORT DIRECTION
L → R
R ← L
T ↓ B
B ↑ T
THRESHOLD VALUE
97
SORTING VALUE
Lightness (Value)
SPATIAL MASKING
NONE
SORT INSIDE
SORT OUTSIDE
RENDER TO CANVAS
EXPORT AS JPEG

8. Conclusion

The GPU-Accelerated Glitch Art Studio serves as a highly efficient solution for creating digital glitch art. By leveraging the parallel processing capabilities of NVIDIA GPUs and CUDA, the system ensures real-time rendering and interactive manipulation of high-resolution images. It addresses the computational limitations of traditional CPU-based sorting methods by offering a high-performance backend, an intuitive frontend, and deep algorithmic optimization.

This project not only enhances the creative workflow for digital artists but also demonstrates the effective application of modern High-Performance Computing technologies such as CUDA C++, dynamic kernel compilation via CuPy, and modern web frameworks. Through various features like spatial masking and multi-channel sorting, the system proves its versatility and power.

From a development perspective, this project offered valuable hands-on experience in integrating low-level parallel computing algorithms with high-level web APIs. It emphasized the importance of thread management, memory optimization, and parallel algorithms like Quick Sort. By working through the system's architecture, the project reinforced foundational knowledge in HPC and software engineering best practices.

Looking ahead, there is significant potential for further enhancement of the system. Future updates could include real-time video processing, advanced AI-driven glitch parameters, and additional hardware-accelerated effects. These features would elevate the system into a comprehensive digital art suite, pushing the boundaries of what is possible with accessible GPU computing.

9. References

[1] NVIDIA Corporation, "CUDA C++ Programming Guide," NVIDIA, 2024.

[2] S. Cook, CUDA Programming: A Developer's Guide to Parallel Computing with GPUs, Morgan Kaufmann, 2012.

[3] CuPy Team, "CuPy: A NumPy-compatible array library accelerated by CUDA," [Online]. Available: https://cupy.dev/.

[4] FastAPI, "FastAPI Documentation," [Online]. Available: https://fastapi.tiangolo.com/.

[5] React, "React Documentation," [Online]. Available: https://react.dev/.

[6] J. Sanders and E. Kandrot, CUDA by Example: An Introduction to General-Purpose GPU Programming, Addison-Wesley Professional, 2010.

[7] OpenCV, "OpenCV Documentation," [Online]. Available: https://docs.opencv.org/.

[8] M. Harris, "Parallel Prefix Sum (Scan) with CUDA," GPU Gems 3, Addison-Wesley, 2007.

[9] Vite, "Vite: Next Generation Frontend Tooling," [Online]. Available: https://vitejs.dev/.

[10] W. H. Press et al., Numerical Recipes: The Art of Scientific Computing, Cambridge University Press, 2007.